

Go runtime internals

Goroutines • G/M/P • scheduler • work stealing • GOMAXPROCS
netpoller • syscalls • stack growth • стоимость goroutine

Главный вопрос

На машине 8 CPU, а приложение создаёт 100 000 goroutine. Кто их выполняет?

Почему Go не создаёт 100 000 OS threads?

Что происходит, когда goroutine ждёт сеть, mutex или syscall?

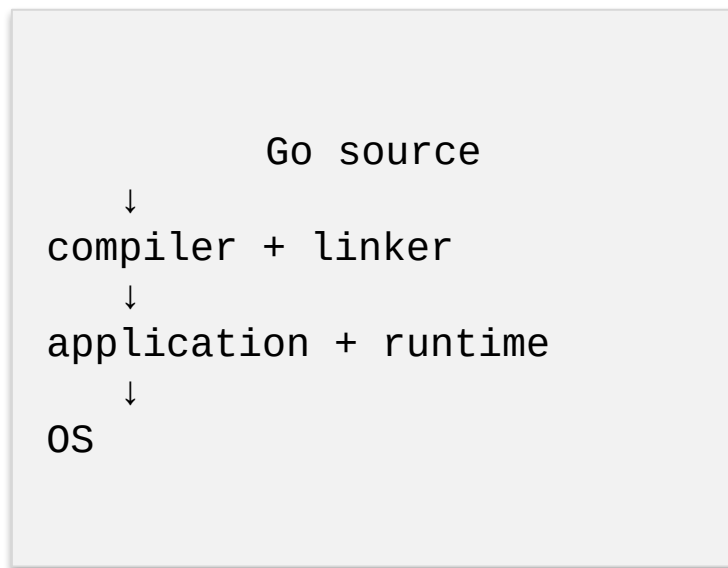
Ответ: Go runtime multiplexes множество goroutine поверх ограниченного набора OS threads.

Go runtime — часть executable

Go компилируется в native executable, но это не «просто машинный код без runtime».

Runtime содержит scheduler, GC, allocator, stack management, timers, netpoller и служебную metadata.

В отличие от JVM runtime не запускается отдельным процессом — он линкован с приложением.



Goroutine: точная ментальная модель

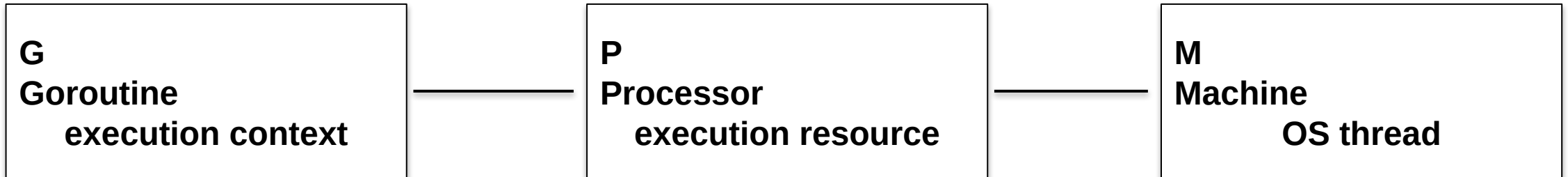
Упрощение: goroutine = «лёгкий thread».

Точнее: goroutine — independently schedulable execution context, которым управляет Go runtime.

Она имеет собственный user stack и scheduler/runtime state.

``go f()`` делает новую goroutine runnable; когда она реально получит CPU, решает scheduler.

G / M / P — три сущности scheduler



G хранит состояние конкретной goroutine.

M физически исполняет инструкции как OS thread.

P нужен M для исполнения Go-кода и несёт per-P runtime resources.

Жизненный цикл G



Большинство из 100 000 goroutine обычно не **RUNNING**: они ждут I/O, timer, channel, mutex или другой ресурс.

M — настоящий OS thread

M создаётся/управляется runtime, но планируется уже kernel scheduler.

Именно M исполняет machine instructions.

Количество M может быть больше GOMAXPROCS: часть M может блокироваться в syscalls/cgo.

Go scheduler выбирает G для M; OS scheduler выбирает, когда сам M получит CPU.

Зачем вообще нужен P?

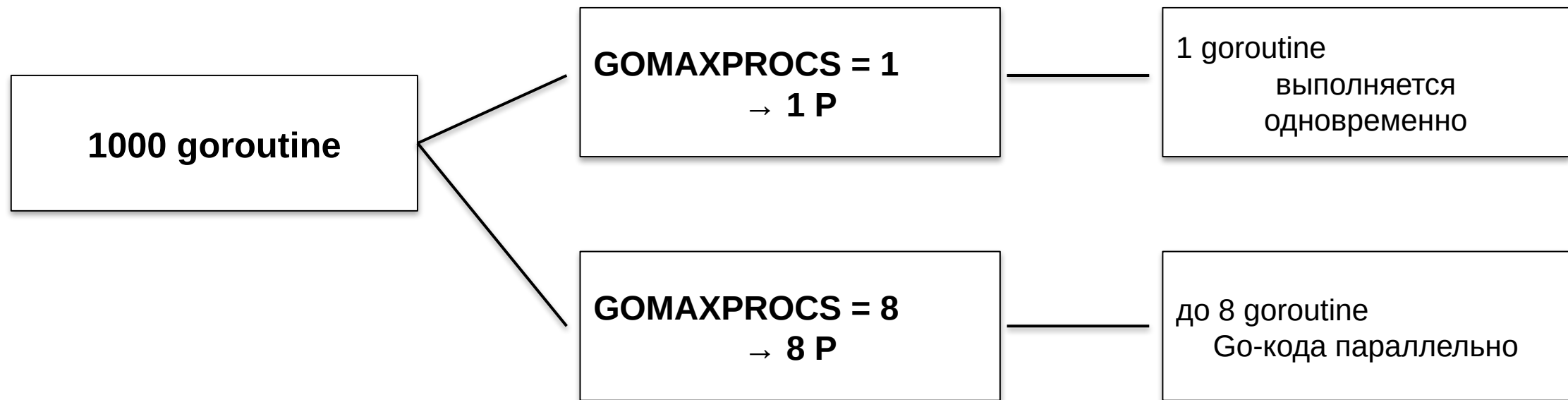
Модель $G \rightarrow M$ недостаточна: runtime нужны per-execution ресурсы, которые нельзя навечно привязать к OS thread.

P содержит локальную run queue и другую per-P runtime state.

M может заблокироваться в syscall и временно потерять P.

Освободившийся P может подхватить другой M и продолжить выполнение Go-кода.

Concurrency ≠ parallelism



GOMAXPROCS

Практическая модель: $GOMAXPROCS \approx \text{количество } P \approx \text{максимальный parallelism Go-кода.}$

Это не количество goroutine и не верхняя граница OS threads процесса.

CPU-bound workload ограничен количеством доступных P/CPU.

I/O-heavy workload может иметь огромное число goroutine при небольшом числе одновременно RUNNING.

GOMAXPROCS в контейнере

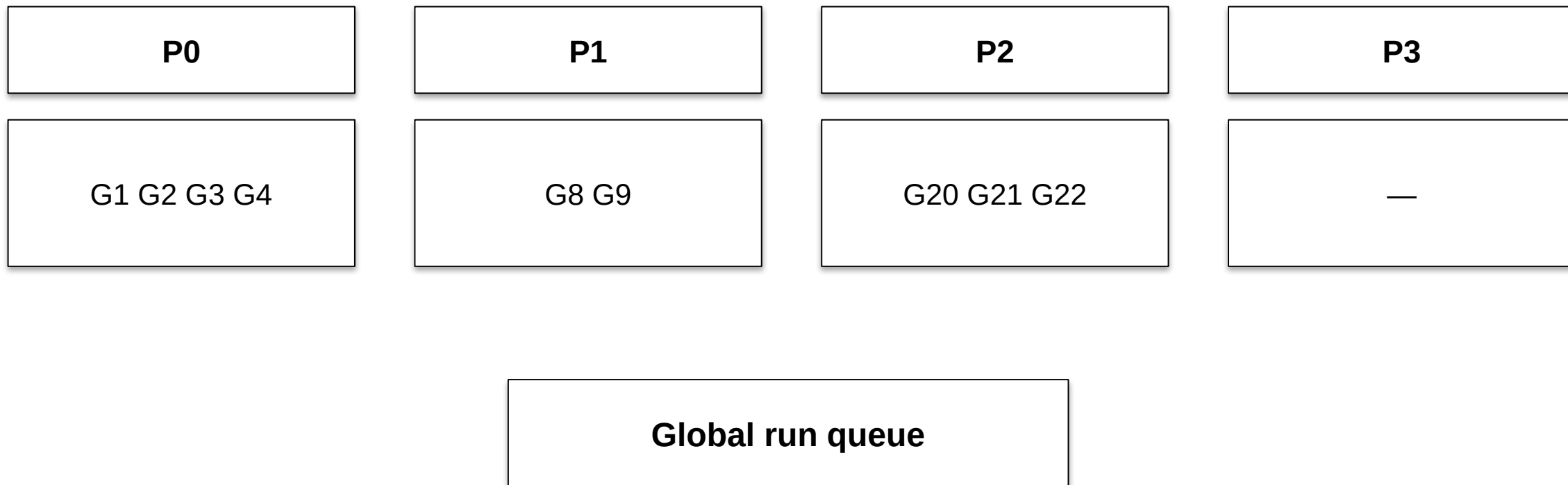
Проблема: host может иметь 128 CPU, а cgroup/container limit — 2 CPU.

Слишком высокий parallelism приводит к quota exhaustion → kernel throttling → ухудшению tail latency.

Современный Go учитывает container CPU limits при выборе default GOMAXPROCS.

Цепочка для backend: GOMAXPROCS → P → M → Linux scheduler → cgroups → p99 latency.

Где ждут RUNNABLE goroutine



Local queues уменьшают contention. Global queue нужна для координации/fairness и для работы, не привязанной к одному P.

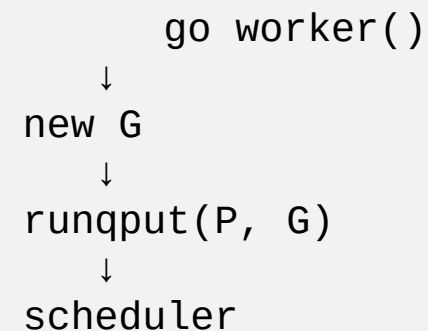
Что делает `go f()`

Compiler генерирует обращение к runtime machinery создания goroutine.

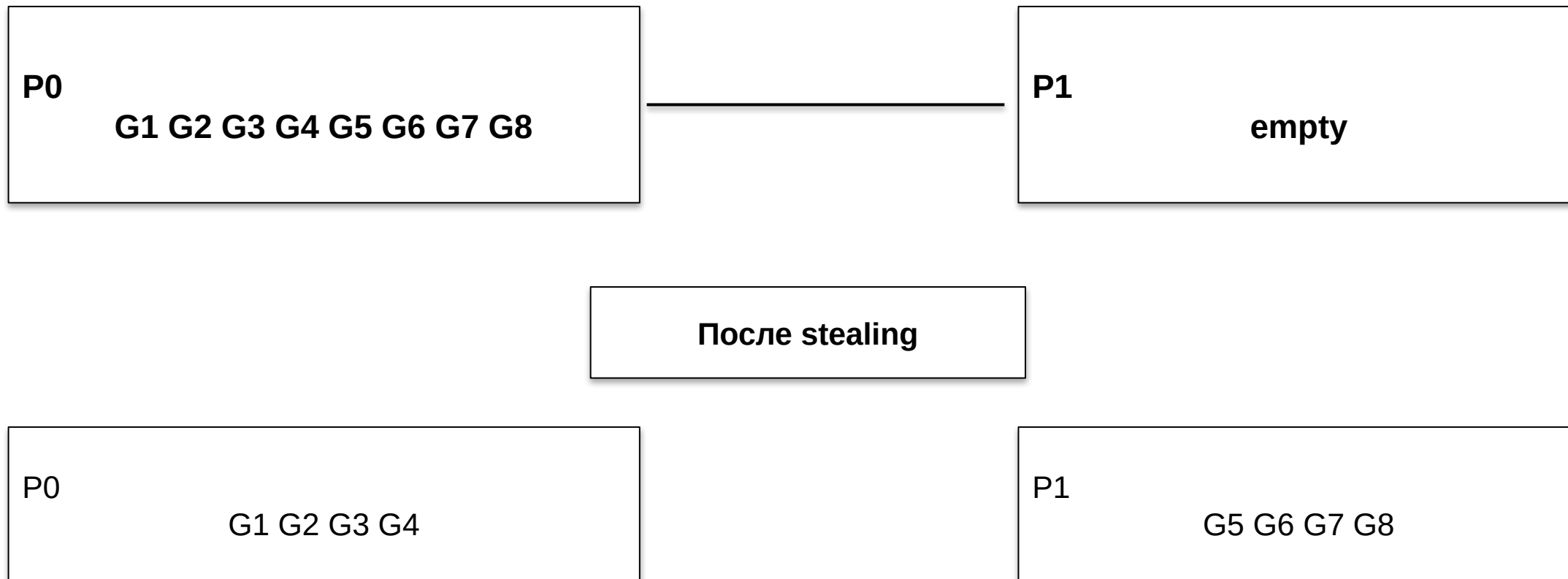
Runtime создаёт новую G и помещает её в runnable queue текущего

В текущей реализации локальная очередь P имеет 256 slots плюс слот

Создание goroutine не означает немедленного запуска функции.



Work stealing: если одному P нечего делать



Почему воровать пачку работы

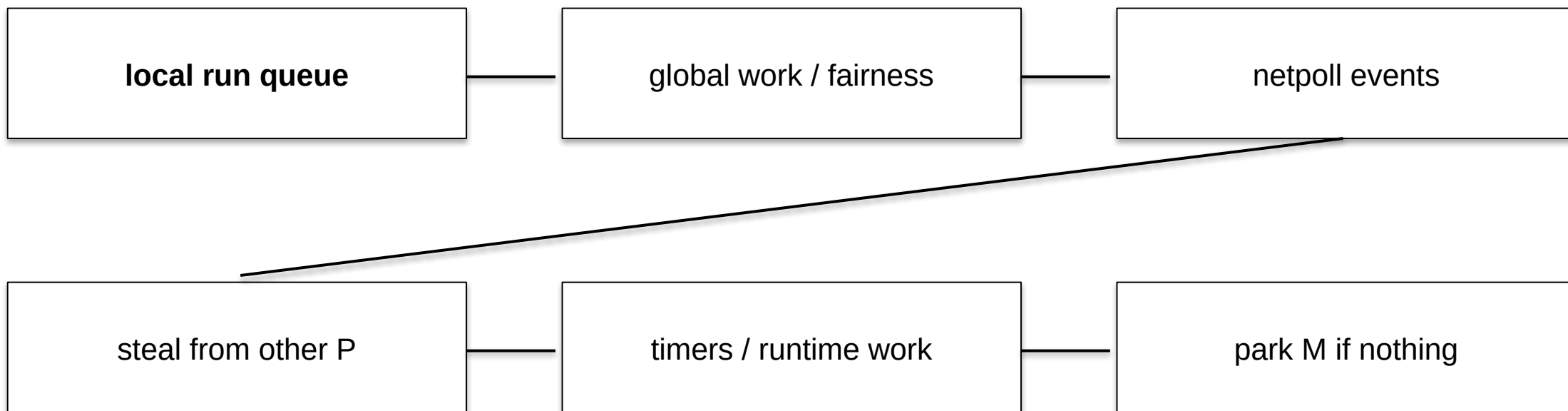
Украсть одну G: быстро закончили → снова идём искать работу → scheduler overhead.

Украсть всё: исходный P сам остаётся пустым.

Классический компромисс work stealing — забрать примерно половину runnable work.

Цель — балансировка без центрального scheduler-босса и глобального lock на каждую операцию.

Упрощённый цикл scheduler



Блокировка goroutine ≠ блокировка OS thread

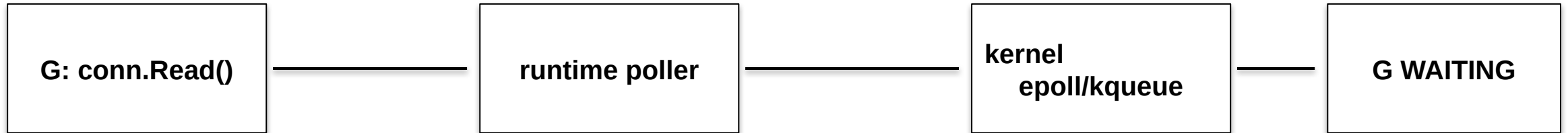
Channel receive, mutex, timer и другие runtime-aware waits могут перевести G в WAITING.

М при этом может продолжить выполнять другую runnable G.

Scheduler сохраняет execution context G и возвращает её в runnable state, когда событие произошло.

Именно это позволяет тысячам ожидающих goroutine не требовать тысячи running threads.

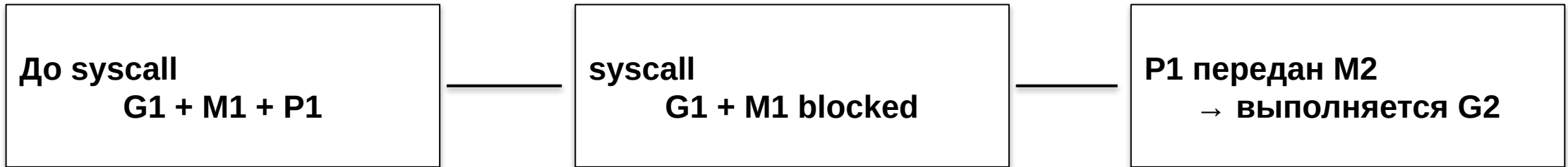
Netpoller: network I/O без thread-per-connection



packet/data ready → G becomes RUNNABLE → scheduler executes it again

Код выглядит синхронным, но ожидание pollable socket интегрировано с scheduler.

Blocking syscall: M действительно может застрять в kernel



После возврата M1 должен снова получить P; если свободного P нет — G1 продолжит позже через scheduler.

Netpoller и blocking syscall — разные механизмы

Pollable network I/O

G ждёт
M свободен для другой G
OS thread не обязан блокироваться

Blocking syscall

G + M могут реально блокироваться в kernel
P отделяется
другой M продолжает Go-код

Всегда задавайте вопрос: goroutine блокируется НА ЧЁМ? От этого зависит runtime-механика.

Preemption: что если G не блокируется добровольно

CPU-bound tight loop не должен бесконечно монополизировать P.

Современный runtime поддерживает asynchronous preemption (начиная с Go 1.14).

В текущей реализации есть внутренний ориентир порядка 10 ms для force-preemption, но это не гарантированный quantum.

Scheduling остаётся best-effort и зависит также от OS scheduler, GC и нагрузки.

Stack goroutine: маленький и динамический



Нет места → allocate larger contiguous stack → copy used frames → fix pointers → continue

Современный runtime может адаптивно выбирать стартовый stack выше минимума по статистике workload.

Почему stack growth — сложнее обычного memсru

После переноса stack его адрес меняется.

Pointers, направленные внутрь старого stack, должны быть скорректированы.

Compiler-generated stack maps помогают runtime понимать pointer layout frames.

Особые runtime structures (defer/panic/channel wait state и др.) тоже требуют корректировки.

У М есть системный stack — g0

User goroutine исполняется на собственном growable stack.

Runtime scheduler и часть внутренних операций выполняются на system stack М (g0).

Это позволяет runtime выполнять операции, которые сами не должны зависеть от обычного user-stack growth/preemption.

Модель: М переключается между user G и своей служебной g0.

Сколько стоит goroutine

Goroutine намного дешевле традиционного OS thread для большого числа waiting tasks, но она не бесплатна.

Цена: G metadata + stack + scheduler work + возможные allocations/captures + GC scanning.

$100\,000 \times 2\text{ KiB} \approx 195\text{ MiB}$ только как грубая нижняя оценка stack memory.

Реальный объём может быть значительно больше из-за heap objects, buffers, timers, sockets и более крупных stacks.

Goroutine leak: scheduler работает правильно, архитектура — нет

```
func process() {  
    ch := make(chan Result)  
  
    go func() {  
        ch <- slowOperation()  
    }()  
  
    return // никто не читает ch  
}
```

1 request → +1 leaked G
1000 requests → +1000 G

Симптомы:

goroutines ↑

memory ↑

GC CPU ↑

latency ↑

Scheduler не создаёт backpressure

Миллион дешёвых goroutine перед PostgreSQL с pool size 20 — всё ещё плохая система.

Scheduler знает, как распределять runnable work; он не знает capacity вашей БД или downstream API.

Нужны bounded concurrency, semaphore, worker pool, connection pool, queue limits или rate limiter.

Дешёвая goroutine — очень дешёвый способ случайно создать огромную очередь прямо в памяти процесса.

Диагностика runtime/scheduler

`runtime.NumGoroutine()` / runtime metrics — динамика числа G.

`pprof` goroutine profile — где и на чём ждут goroutine.

`go tool trace` — lifecycle G, scheduler latency, network wait, syscalls, GC, P/M activity.

`GODEBUG=schedtrace=1000,scheddetail=1` — snapshot scheduler state.

Ключевой вопрос: CPU bottleneck, network wait, syscall, mutex/channel, connection pool или leak?

Java-мост: Thread / Virtual Thread / Goroutine

Java platform Thread
≈ OS thread

Java virtual thread
user-mode task → carrier
thread

Go goroutine
G → Go scheduler → M

Общая идея: дешёвый logical execution context не делает CPU, DB connections и downstream capacity бесконечными.

Финальная модель

G G G G G G G G

↓
local/global runnable queues

P0 P1 P2 P3

↓ ↓ ↓ ↓
M0 M1 M2 M3

↓
OS scheduler

↓
CPU

network: G → netpoller → WAITING → RUNNABLE

syscall: G+M → kernel, P → another M

stack: no space → grow/copy/fix → continue

Что важно запомнить

$G \neq \text{OS thread}$. G — runtime execution context; M — OS thread; P — ресурс исполнения Go-кода.

GOMAXPROCS ограничивает parallelism Go-кода через число P , но не число goroutine или M .

Local run queues + work stealing уменьшают contention и балансируют runnable work.

Netpoller позволяет тысячам сетевых goroutine ждать I/O без thread-per-connection.

Blocking syscall может удерживать M , поэтому P отделяется и продолжает работу с другим M .

Growable stacks уменьшают стартовую стоимость G , но goroutine всё равно потребляет память и GC/scheduler resources.

Scheduler решает scheduling, но не backpressure и не lifecycle ваших ресурсов.